

C语言基础知识补充

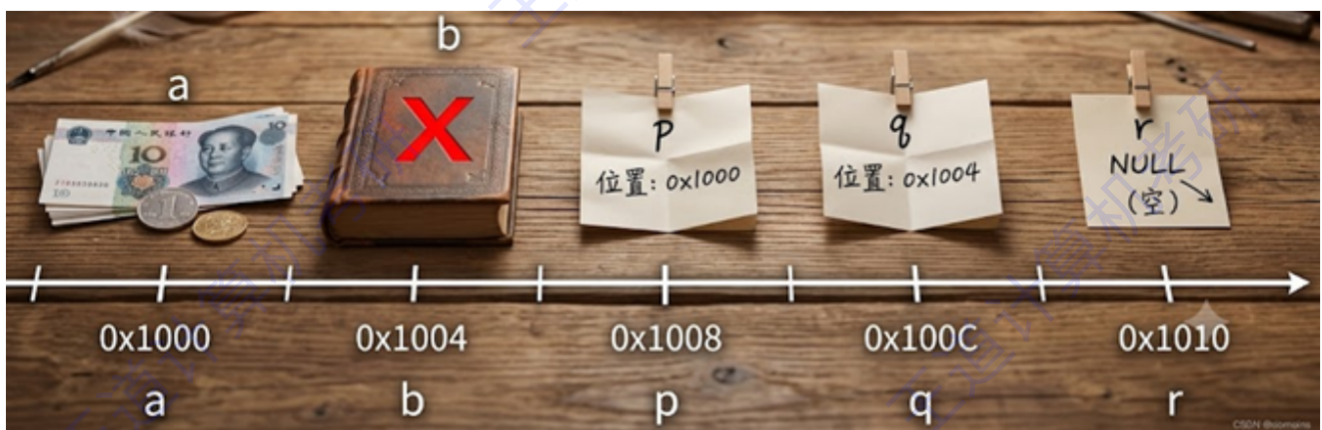
指针是什么

很多同学记住了 `int *p` 这样的写法，但不知道 `*` 到底在干什么。

概念	包类比
普通变量 <code>int a = 10</code> 、 <code>char b = 'x'</code>	<code>a=10</code> 块钱， <code>b=</code> 一本叫' <code>x</code> '的书
指针变量 <code>int *p</code>	一张纸条，上面记着某个包的位置
<code>*p</code>	根据位置 <code>p</code>

代码示例：

```
int a = 10;           // 一堆钱，共 10 块钱
char b = 'x';         // 一本叫 'x' 的书
int *p = a;           // 一张纸条，记录的是钱的位置
char *q = b;          // 一张纸条，记录的是书的位置
struct LNode *r = NULL; // 一张纸条，记录的是包的位置
```



结构体是什么

很多同学背了 `struct LNode { int data; struct LNode *next; };`，但完全不懂这段代码在说什么。

`int` 变量是一堆钱，`char` 变量是一本书。如果我们想把某些物品打包装一起就需要使用 `struct`。

结构体就是定制一款包，刚好能装下这些东西（先不考虑对齐问题）。包中会有多个口袋，每个口袋装一个东西，比如说下面的爱马仕菜篮子（Picotin Lock 18）



代码拆解：

```
struct LNode           // 1. 宣布：一款叫 struct LNode 的包
{
    int data;           // 2. 口袋1：放一堆钱（一个整数）
    struct LNode *next; // 3. 口袋2：放一张纸条，记录了某个同款包的位置
};
```

如何使用这样的包呢，可以直接使用 LNode 吗？不行，首先得买一个包A，也就是：

```
struct LNode A;         // 定义一个变量A，也就是买了一款叫 struct LNode 的包
struct LNode *p;        // *p是一个包，而p是一个指向包的指针（纸条）
```

这个包里面有部分，分别是 data (int 钱) 和 next (struct LNode * 纸条)。要查找某部分需要使用 "包名字.data" 和 "包名字.next"。而如果我们手上只有纸条 p，就需要根据纸条的线索去查找，也就是使用 ->（有个查找和推测的过程）。所以大家在使用的时候一定要问自己一句：我拥有的的是什么？，区分A、p是结构体还是指针，结构体使用 .，指针使用 ->：

```
A.data = 10;
A.next = NULL;
p->data = 10;
p->next = NULL;
```

但是我们发现，每次定义（买包）都需要使用 `struct LNode`，比如这款包叫“人造的爱马仕菜篮子”太长太难听了，所以我们给他改个名字，叫 原神 爱马仕。用法：

```
typedef 原名称 别名1, 别名2, ...;
typedef struct LNode LNode;           // 把 struct LNode 取别名 LNode
```

那如果把以下两条指令结合：

```
struct LNode           // 1. 宣布：一款叫 struct LNode 的包
{
    int data;           // 2. 口袋1：放一堆钱（一个整数）
    struct LNode *next; // 3. 口袋2：放一张纸条，记录了某个同款包的位置
};
typedef struct LNode LNode;           // 把 struct LNode 取别名 LNode
```

就变成

```
typedef struct LNode           // 1. 宣布：一款叫 struct LNode 的包
{
    int data;           // 2. 口袋1：放一堆钱（一个整数）
    struct LNode *next; // 3. 口袋2：放一张纸条，记录了某个同款包的位置
} LNode;
```

接下来就只需要这样定义了：

```
LNode A;           // 定义一个变量A，也就是买了一款叫 struct LNode 的包
LNode *p, *q;       // *p是一个包，而p是一个指向包的指针（纸条）
```

我还是觉得有点丑，每次定义指针都需要加一个 `*`，能不能优化一下，给一个指针类型（纸条也取名，一类指向这款包的纸条也有名字 `linklist`）

```
typedef struct LNode           // 1. 宣布：一款叫 struct LNode 的包
{
    int data;           // 2. 口袋1：放一堆钱（一个整数）
    struct LNode *next; // 3. 口袋2：放一张纸条，记录了某个同款包的位置
} LNode, *linklist;
```

修改之后

```
LNode A;           // 定义一个变量A，也就是买了一款叫 struct LNode 的包
linklist p,q;       // *p是一个包，而p是一个指向包的指针（纸条）
```

链表

考试中的链表默认有头结点，链表一般不会给出元素个数，遍历需要通过 while 循环，当碰到空NULL的时候就已遍历完。

```
void traverse(LNode *L)
{
    LNode *p = L->next;    // p 从第一个元素 L->next 开始，若无头结点则是 L
    while (p != NULL)
    {
        visit(p->data);
        p = p->next;
    }
}
```

return 用法

函数就像一个管理员。你给他指令（参数），他在里面忙活一阵，最后 return 做两件事：

1. 立刻下班——后续指令全部不执行
2. （可选）递出来一样东西——可能是钱、包、纸条

注意，返回的类型和函数前写的类型需要完全相同。

return 本质上是提前结束函数并返回值（可以理解成提前交卷）。

提前结束，递出类型 int / float / char 等

```
int add(int a, int b)    // 管理员收到两堆钱 a 和 b
{
    return a + b;        // 把两堆钱加起来，递给你结果，然后下班
}
```

void 函数可以写 `return;` 来提前收工，不返回值：

```
void PreOrder(BiTNode *T)
{
    if (T == NULL) return;    // 给的纸条是空的 → 直接下班
    visit(T);                // 否则继续干活
    PreOrder(T->lchild);
    PreOrder(T->rchild);
}
```

提前结束，返回 NULL（属于指针类型）

```
BiTNode* BST_Search(BiTNode *T, ElemType x)
{
    BiTNode *p = T;
    while (p != NULL)
    {
        if (x == p->data)
            return p;        // 找到了目标包的位置，把纸条递给你
        if (x < p->data)
            p = p->lchild;
        else
            p = p->rchild;
    }
    return NULL;            // 没找到，递一张空白纸条
}
```

不提前结束，会返回默认值

```
void traverse(LNode *L)    // 收到一张纸条 L
{
    LNode *p = L;
    while (p != NULL)
    {
        visit(p->data);    // 打开包查看
        p = p->next;       // 拿纸条去下一个包
    }
}
```

```
// 所有包都查看过了 → 默默下班，不递东西
}
```

多条路，每一条都有 return，但只会经过一条路

```
int find(int a[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (a[i] == x)
            return i;           // 走这条路：找到了，下班
    return -1;                  // 走那条路：没找到，下班
}
// 不管走哪条，一定会 return，不会出现管理员站门口不走的情况
```

函数的参数

顺序表

```
void traverse(ElemType data[], int n)
{
}
```

链表

```
void traverse(LNode * L)
{
}
```

树

```
void PreOrder(BiTNode * T)
{
}
```

图

```
int IsExistEL(MGraph G)
{
}
```

我们需要知道每一个参数是做什么的、是什么类型，在使用参数时一定要和参数完全匹配。比如说要使用邻接矩阵中的二维数组，应该是 `G.edges[i][j]`。

& 怎么用

取地址运算符最常见的用途是为指针变量赋值，使指针变量指向某个特定的内存地址（一般是指向一个变量）。

建议考试时永远不使用，当你在函数参数中需要使用&时，可以试着用全局变量来实现

缩进规范

同层级的括号需要对应

```
void traverse(LNode *L)           // 第 0 层：站在工作区大门
{
    LNode *p = L;                // 第 1 层：进了大门
    while (p != NULL)            // 第 1 层
    {                             // { 和 } 同一层对齐
        visit(p->data);           // 第 2 层：进了 while 的区域
        p = p->next;              // 第 2 层
    }                             // 第 1 层：出了 while
}                                 // 第 0 层：出了大门
```

错误（不要这样）：

```
void traverse(LNode *L){
    LNode *p = L;
    while(p!=NULL){
        visit(p->data);
        p=p->next;
    }
}
```

改正带

以后禁止使用改正带，防止大家考试时用

